

Kubernetes CNI

Kubernetes 공식문서 중 [Services, Load Balancing, and Networking](#) 이라는 문서가 있습니다.

여기에는 The Kubernetes network model 이라는 부분이 있는데, 여기서 Pod 간의 네트워킹에 대해 다음과 같이 정의하고 있습니다.

All pods can communicate with all other pods, whether they are on the same [node](#) or on different nodes. Pods can communicate with each other directly, without the use of proxies or address translation (NAT).

간단히 요약하면, 모든 Pod는 클러스터 전체에서 고유한 IP를 가지며, 같은 노드든 다른 노드든 Pod끼리 **Proxy나 NAT 없이** 직접 통신할 수 있어야 합니다.

사실 같은 노드 내에서는 Proxy나 NAT 없이 통신이 불가능할 것 같지는 않습니다. 하지만 서로 다른 노드 간에 어떻게 Proxy나 NAT 없이 통신할 수 있는지 궁금해졌고, 이 부분에 대해 알아보려고 합니다.

1. Kubernetes Cluster Networking

Kubernetes 공식문서 중 [Cluster Networking](#) 이라는 문서에서, Cluster Networking 에는 Kubernetes가 해결해야 하는 4가지의 Networking Problems 에 대해 다음과 같이 정의하고 있습니다.

1. Highly-coupled container-to-container communications: this is solved by [Pods](#) and `localhost` communications.
2. Pod-to-Pod communications: this is the primary focus of this document.
3. Pod-to-Service communications: this is covered by [Services](#).
4. External-to-Service communications: this is also covered by [Services](#).

한국어로 번역하면 다음과 같습니다.

1. 컨테이너 ↔ 컨테이너: 같은 Pod 안의 컨테이너끼리 통신
2. Pod ↔ Pod: 클러스터 내부 워크로드 간 통신
3. Pod ↔ Service: 변하는 Pod 집합에 안정적인 이름과 주소로 접근
4. 외부 ↔ Service: 클러스터 외부 트래픽을 애플리케이션으로 전달

우선 이번 글에서는 Pod ↔ Pod 통신에 대해 알아보도록 하겠습니다.

2. Pod IP 할당

일반적인 Docker 환경에서는 컨테이너가 호스트의 NAT 뒤에 숨는(?) 경우가 많습니다. 반면 Kubernetes는 각 Pod에 독립적인 IP를 할당합니다.

예를 들어 다음과 같은 클러스터가 있다고 해보겠습니다.

대상	IP 예시
Node A	192.168.10.11
Node B	192.168.10.12
Pod A (Node A)	10.244.1.10
Pod B (Node B)	10.244.2.20

이때 Pod A가 Pod B에 접속하면, Kubernetes의 철학에 따라 애플리케이션 관점에서는 다음과 같이 표현할 수 있어야 합니다. 여기서 중요한 점은 각 Pod의 원본 IP가 유지되어야 한다는 점입니다.

```
10.244.1.10 → 10.244.2.20
```

3. CNI

Kubernetes에서는 Pod-Pod 간에 NAT를 사용하지 않고 고유한 IP를 통해 서로 통신해야 한다는 것을 배웠습니다. 그럼 이것을 어떻게 달성하는 거죠?

Kubernetes는 기본적으로 Pod에 IP를 할당하고 노드 간 경로를 만드는 구체적인 방식을 고정하지 않습니다. 이것은 CNI의 역할입니다.

CNI(Container Network Interface)는 모든 Pod에 IP를 할당하고 노드 간 경로를 만드는 구체적인 방식을 구현하는 인터페이스입니다. CNI는 Pod가 생성될 때 주로 다음 작업을 수행합니다.

1. Pod의 Network Namespace에 Network Interface 연결
2. Pod IP 할당
3. Pod에서 노드로 나가는 경로 생성
4. 노드 간 Pod CIDR 라우팅 또는 Overlay 터널 생성

다만, 어떤 CNI를 사용하느냐에 따라 Network 구현 방식은 모두 다릅니다. 대표적인 방법들에 대해 공부해보면서, 각각의 방법으로 어떻게 다른 노드의 Pod로 도달하는지 한 번 보겠습니다.

4. 서로 다른 노드의 Pod에 도달하는 방식 정리

CNI마다 구현 세부 사항은 다르지만, 다른 노드에 있는 Pod로 패킷을 전달하는 방식 중 많이 쓰이는 3가지에 대해 알아보겠습니다.

1. 직접 L3 라우팅: Underlay 네트워크가 Pod CIDR의 경로를 알고 있음
2. IPIP Overlay: Pod IP 패킷을 Node IP 패킷 안에 넣어 전달
3. VXLAN Overlay: Pod의 L2 프레임을 UDP 기반 VXLAN 패킷으로 감싸 전달

세 방식은 모두 최종적으로 다음 Kubernetes 네트워크 모델을 만족시킨다는 공통점이 있습니다.

```
Pod A (10.244.1.10) → Pod B (10.244.2.20)
```

즉 목적지 Pod B가 보는 요청의 출발지 IP는 Node A의 IP가 아니라 Pod A의 IP여야 합니다.

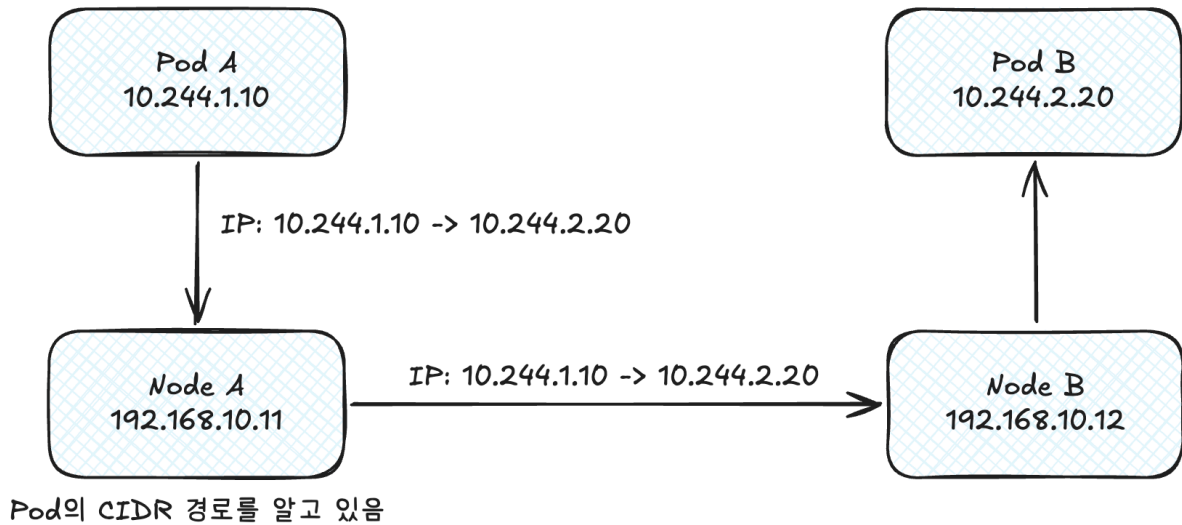
4.1. 직접 L3 라우팅

Underlay 네트워크가 **Pod IP** 대역의 경로를 알고 있다면, **Overlay** 없이 원본 Pod IP 패킷을 그대로 전달할 수 있습니다. 이 방식을 사용하는 대표적인 CNI가 Calico로, Calico는 **BGP**나 정적 라우팅을 사용해 이러한 구성을 지원합니다.

Calico 의 공식문서 중 **Overlay Networking** 에서는 다음과 같이 설명하고 있습니다.

Networks become aware of workload IP addresses through layer 3 routing techniques like static routes or BGP route distribution, or layer 2 address learning. As such, they can route unencapsulated traffic to the right host for the endpoint that is the ultimate destination.

이를 한국어로 번역해보면, Calico는 Underlay가 Workload IP 주소를 라우팅할 수 있다면 encapsulation 없이 통신할 수 있으며, **BGP route distribution**은 Underlay에 Workload IP 경로를 알리는 대표적인 방법이라고 합니다.

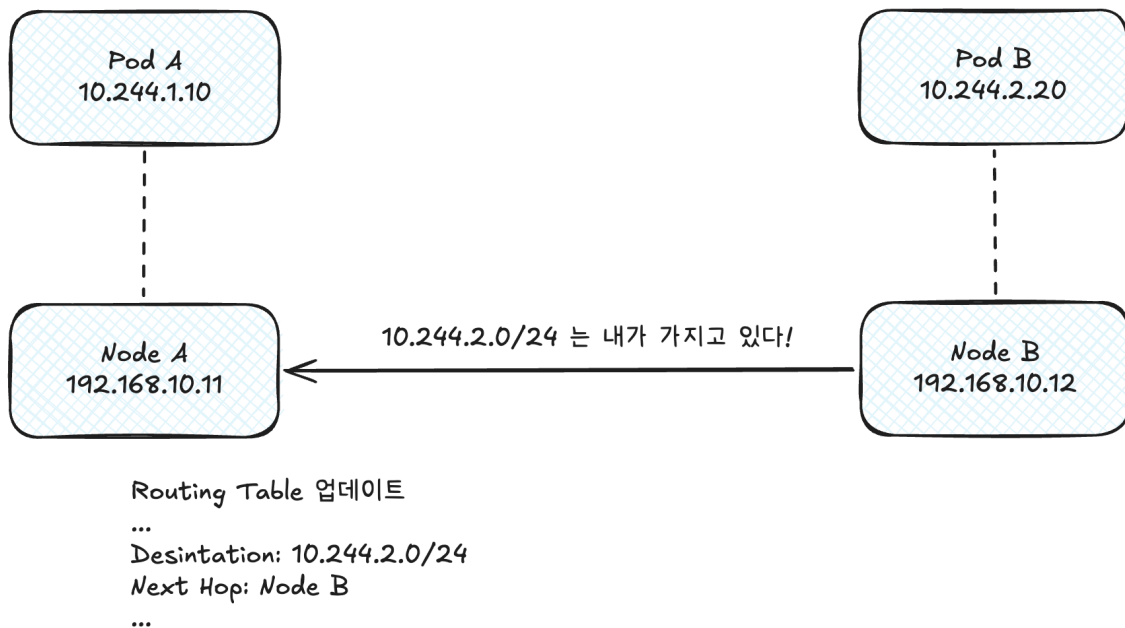


이 방법의 경우, Node A에서 Node B까지 이동하는 동안에도 IP 헤더의 출발지와 목적지는 Pod IP 그대로입니다. 다만, 이 방법을 사용하기 위해서는 사전에 라우팅 정보를 노드가 알고 있어야 합니다.

예를 들어, Node A와 Underlay는 다음 사실을 알아야 합니다.

"10.244.2.20/24 대역은 Node B 방향으로 가야 한다."

BGP를 사용하는 경우, Node B는 자신이 보유한 Pod CIDR를 BGP로 알립니다. 그리고 이를 받은 Node A는 라우팅 정보를 업데이트 합니다.



직접 L3 라우팅 방법은 Underlay가 Pod의 CIDR를 학습하는 방식으로 장단점은 명확합니다.

장점

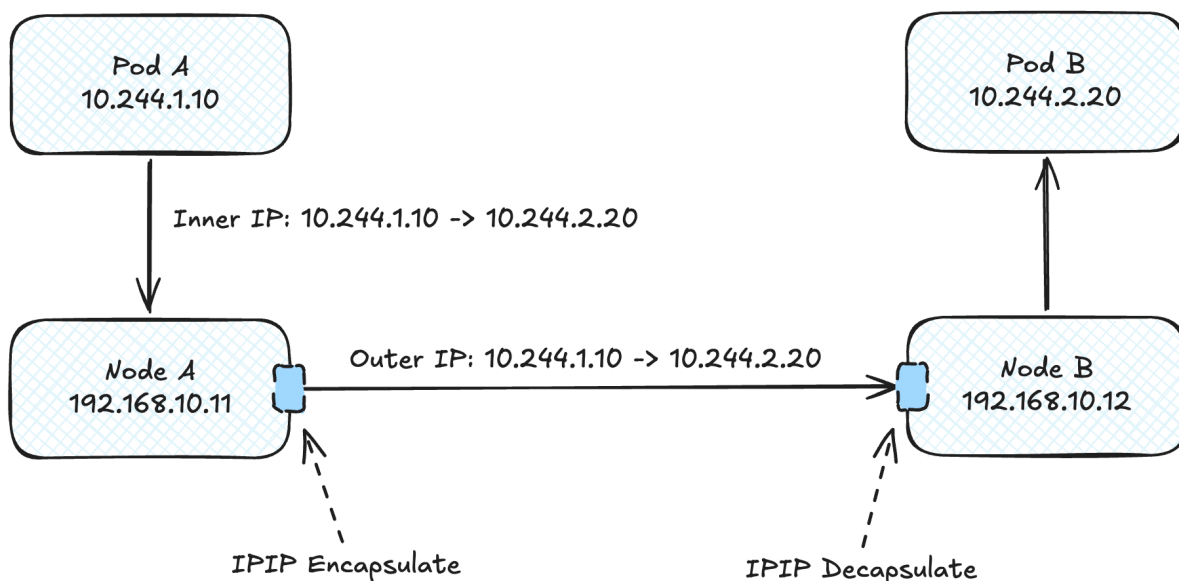
- 캡슐화 헤더가 없어 MTU 성능 측면에서 유리
- 패킷 구조가 단순
단점
- Underlay에 Pod CIDR 경로를 설정 및 배포해야 함
- 네트워크 장비 또는 클라우드 라우팅 제약이 있을 수 있음
- 대규모 환경에서는 BGP / Route Reflector 설계가 필요할 수 있음

4.2. IP-in-IP (IPIP)

두 번째는 IP-in-IP (줄여서 IPIP) 방식입니다. Calico에서 널리 사용되는 방식이며, Underlay가 Pod CIDR를 직접 알 필요가 없다는 장점이 있습니다.

IPIP 방식을 사용하게 되면, Underlay가 알아야 하는 것은 오직 Node IP간 경로입니다. Pod IP 대역인 10.244.0.0/16은 Underlay에 노출할 필요가 없어집니다.

Node A (192.168.10.11) → Node B (192.168.10.12)



위 예시에서 Node A가 캡슐화한 패킷은 다음과 같습니다.

Outer IP header

Source IP:	192.168.10.11	(Node A)
Destination IP:	192.168.10.13	(Node B)
Protocol:	4	(IP-in-IP)

Inner IP header

Source IP:	10.244.1.10	(Pod A)
Destination IP:	10.244.2.20	(Pod B)
Protocol:	TCP	

TCP header + application data

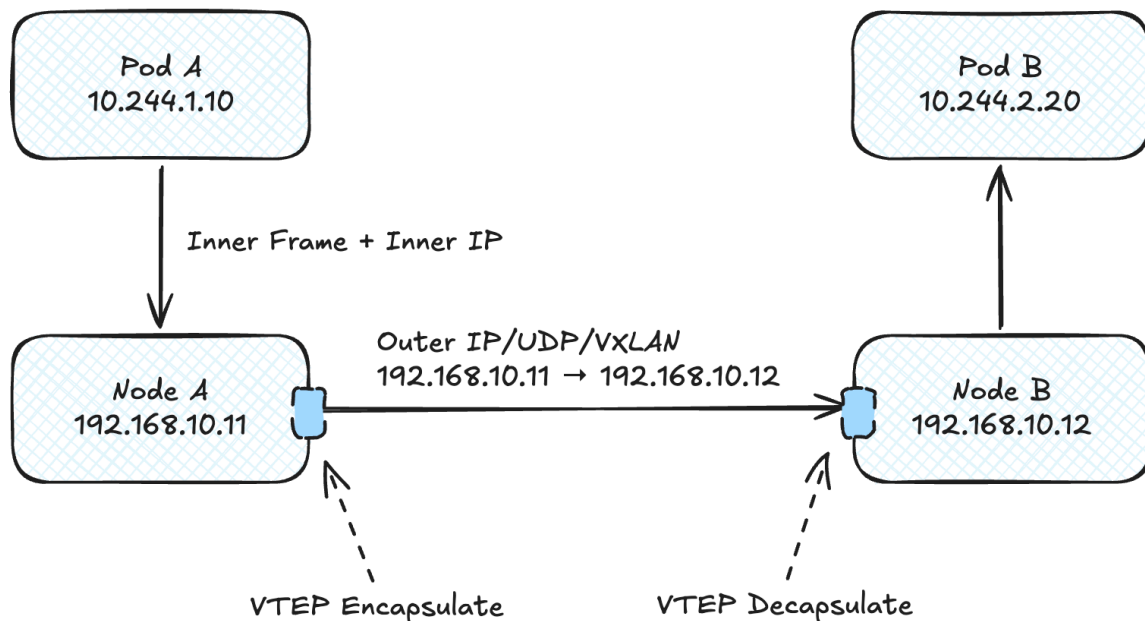
이 캡슐화한 패킷이 Node B에 도착하면, IPIP Decapsulate 과정을 통해서 Outer IP를 벗기고, 그 안에 있는 Inner IP를 통해 요청을 보낸 Pod IP와 목적지 Pod IP를 알 수 있게 됩니다.

IPIP 방식은 Underlay에서 Pod CIDR를 알 필요가 없어졌지만, 각 패킷마다 IPv4 헤더를 하나 더 추가하므로 MTU가 감소하는 단점이 있습니다.

4.3. VXLAN Overlay

세 번째는 VXLAN(Virtual eXtensible LAN) Overlay 방식입니다. Flannel이 대표적이며, Calico와 Cilium도 VXLAN 모드를 지원하고 있습니다.

앞서 보았던 IPIP가 IP 패킷을 다시 IP 패킷으로 감싼다면, VXLAN은 내부 Ethernet Frame을 VXLAN 헤더와 UDP/IP 패킷으로 감쌉니다.



기존의 패킷을 감싼다는 점에서는 IPIP 방식과 동일합니다. 다만 이 두 방법의 차이는 무엇을 감싸고, 어떤 프로토콜로 운반하느냐에서 차이가 있습니다.

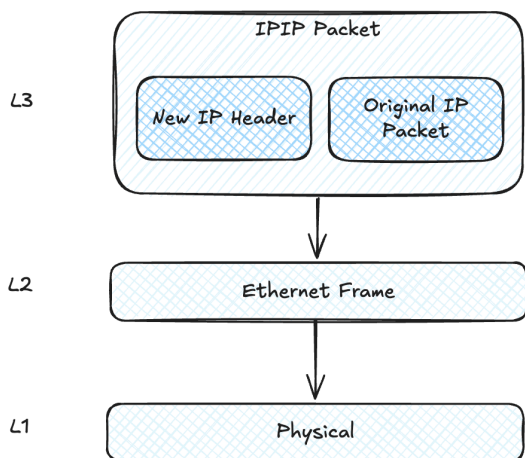
IPIP는 원본 IP 패킷 하나만을 캡슐화 대상으로 삼습니다. 즉 감싸는 헤더도 새로운 IP 헤더 하나뿐이라, 구조가 단순하고 오버헤드도 작습니다. 하지만 IP 헤더 안에는 MAC 주소나 VLAN 태그 같은 L2 정보가 들어갈 자리가 없기 때문에, IPIP는 오직 L3(라우팅) 수준의 터널링만 가능합니다.

반면 VXLAN은 원본 패킷의 IP 헤더뿐 아니라 그 패킷을 감싸고 있던 Ethernet Frame 전체를 캡슐화 대상으로 삼습니다. 이 Ethernet Frame을 VXLAN 헤더로 감싸고, 그 위에 다시 UDP/IP 헤더를

씩워 실제 물리 네트워크로 내보냅니다.

라고 하는데... 사실 이 부분은 아직도 정확하게 이해했는지 모르겠습니다. VTEP가 보낼 예정인 Ethernet Frame을 다시 새로운 UDP/IP 패킷에 집어넣어서 보내는 것 같은데... 왜 그러는거죠?? 어떤 장점이 있는건지 잘 모르겠네요..

IPIP



VXLAN

